

적응신호처리 HW03 Report

20261099 양예진

1. 입력 음성 확인

입력 파일 `a.wav` 를 확인한 결과 다음과 같은 특성을 가지고 있었다.

- 남성 화자의 영어 발화
- Sampling rate: 8000 Hz
- 1 channel (mono)
- 총 샘플 수: 18800개
- 재생 시간: 2.35초
- `soundfile` 로 읽었을 때 데이터 타입: `float64`

이번 과제에서는 입력 음성으로부터 10차 lattice predictor를 구성하고, prediction error와 white noise를 각각 all-pole lattice filter에 입력하여 음성을 합성했다. 먼저 과제에서 요구한 대로 음성 전체를 하나의 모델로 처리한 뒤, 그 결과를 청크별로 모델링한 경우와 비교했다.

2. 전체 음성 모델링

Reflection coefficient 계산

전체 음성의 자기상관함수로부터 Levinson-Durbin 알고리즘을 이용해 reflection coefficient κ_m 을 구했다. 자기상관을 계산할 때는 입력의 평균을 뺀고, 모든 lag에서 전체 샘플 수로 나누었다.

각 차수의 reflection coefficient와 prediction error power는 다음과 같다.

$$\kappa_m = -\frac{\Delta_{m-1}}{P_{m-1}}, \quad P_m = P_{m-1}(1 - \kappa_m^2)$$

계산한 계수들을 소수 셋째 자리까지 나타내면 다음과 같다.

$$[\kappa_1, \dots, \kappa_{10}] \approx [-0.609, -0.324, 0.113, 0.217, 0.052, \\ -0.009, -0.107, 0.225, 0.146, -0.112]$$

P_{10} 은 약 0.00102244였으며, 이렇게 구한 계수들을 음성의 처음부터 끝까지 동일하게 적용했다.

Lattice predictor 적용

Lattice predictor에서는 forward prediction error와 backward prediction error를 다음과 같이 계산했다.

$$f_m(n) = f_{m-1}(n) + \kappa_m b_{m-1}(n-1)$$

$$b_m(n) = b_{m-1}(n-1) + \kappa_m f_{m-1}(n)$$

$f_0(n) = b_0(n) = u(n)$ 으로 두고, 시작 전 지연 상태는 0으로 설정했다. 위 식을 10차까지 적용한 결과, 원본의 RMS는 0.04618이고 $f_{10}(n)$ 의 RMS는 0.03198이었다.

All-pole lattice filter 적용

합성에서는 같은 계수를 사용하여 다음 식을 10차부터 역순으로 적용했다.

$$f_{m-1}(n) = f_m(n) - \kappa_m b_{m-1}(n-1)$$

첫 번째 입력으로는 앞에서 구한 $f_{10}(n)$ 을 사용했다. 두 번째 입력은 평균 0, 분산 P_{10} 인 white noise로 생성했으며, 같은 결과를 얻을 수 있도록 난수 시드를 3으로 고정했다. 두 출력은 각각 `syn_f10.wav`, `syn_noise.wav`로 저장했다.

3. 전체 모델의 결과 분석

입출력 신호 비교

`syn_f10.wav`은 청취 결과 원본과 거의 같은 신호로 복원되었다. 저장 전 최대 절댓값 오차는 약 1.11×10^{-16} 이었고, 16-bit PCM WAV로 저장한 결과는 원본과 모든 샘플이 일치했다.

반면 `syn_noise.wav`을 직접 들어 본 결과, 원래 발화를 알아들을 수 없고 잡음만 들렸다.

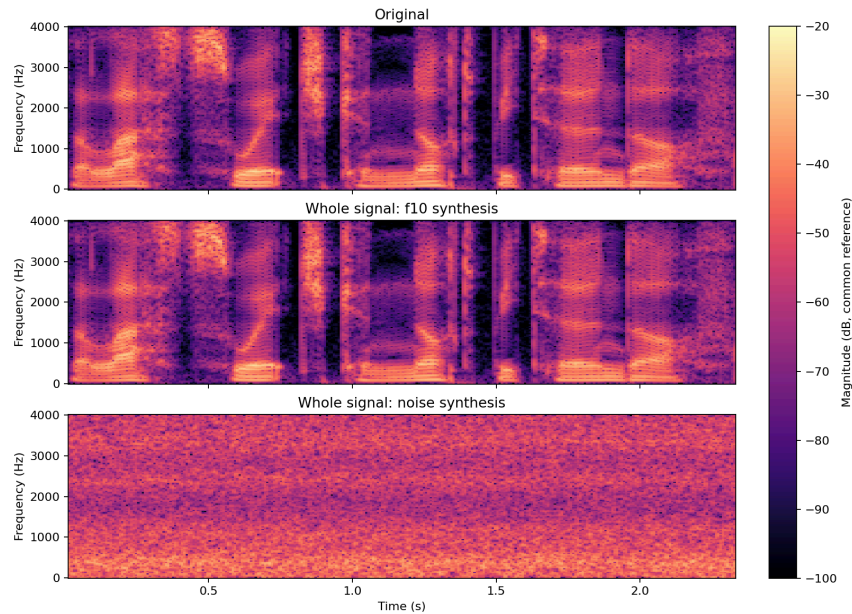


그림 1. 세 신호를 256-sample Hann window와 75% overlap으로 STFT 분석

그림 1에서 원본과 prediction error를 입력한 출력은 동일하게 나타났다. 하지만 white noise를 입력한 출력은 원본과 달리 비슷한 주파수 분포가 계속 이어졌다. 저주파 쪽에 에너지가 모이는 경향은 있지만, 원본에서 보이는 발화와 쉼에 따른 변화는 잘 나타나지 않았다.

잡음 출력의 RMS는 0.04609로 원본의 0.04618과 거의 같았다. 즉, 전체적인 신호의 크기는 비슷하지만 에너지가 나타나는 위치와 시간에 따른 변화가 달랐다.

전체 모델의 한계

Wiener-Hopf 방정식을 직접 풀어 얻은 계수로 `scipy.signal.lfilter`를 적용했다. 이를 lattice 출력과 비교한 결과, prediction error의 최대 차이는 약 1.67×10^{-16} , 잡음 합성 출력의 최대 차이는 약 1.11×10^{-16} 이었다. 즉, 잡음처럼 들리는 결과는 구현의 오류가 아니었다.

그렇다면 같은 모델을 사용했는데도 $f_{10}(n)$ 을 입력했을 때는 왜 원본이 복원되는지 생각해 보았다. 분석 필터를 $A(z)$ 라고 하면 prediction error에는 다음 관계가 성립한다.

$$F_{10}(z) = A(z)U(z), \quad \hat{U}(z) = \frac{F_{10}(z)}{A(z)} = U(z)$$

Prediction error에는 원본을 복원하는 데 필요한 정보가 남아 있기 때문에, 같은 계수를 사용하는 합성 필터를 통과시키면 원본으로 돌아갈 수 있다. 하지만 이를 white noise로 바꾸면 원래 excitation에 담겨 있던 시간적인 정보는 사라진다.

실제 음성은 발음에 따라 포먼트와 에너지가 달라지지만, 전체 음성에서 구한 하나의 고정된 필터로는 이러한 변화를 따라갈 수 없다. 따라서 $f_{10}(n)$ 을 이용한 복원은 가능하더라도, 잡음 입력에서 음성의 변화를 재현하기에는 전체 모델만으로 부족하다고 생각할 수 있다. 따라서 청크별로 모델링하는 방식으로 다시 구해 비교했다.

4. 청크별 모델링

음성을 20 ms씩 나누고, 각 구간에서 κ_m 과 P_{10} 을 다시 계산했다. Sampling rate가 8000 Hz이므로 한 청크는 160 samples이며, 총 118개 청크가 만들어졌다. 마지막 청크는 남은 80 samples를 사용했다. 청크는 서로 겹치지 않도록 했으며 별도의 window는 곱하지 않았다.

Lattice predictor와 all-pole lattice filter에는 앞에서 작성한 함수를 그대로 사용했다. 각 필터의 자연 상태는 따로 유지하고, 청크가 바뀌어도 초기화하지 않고 다음 청크로 이어 주었다.

White noise를 생성할 때는 전체 모델과 같은 표준정규 난수 수열을 사용하되, 각 청크의 $\sqrt{P_{10}}$ 을 곱했다. 따라서 청크 안에서는 white noise이지만, 청크마다 입력 잡음의 세기가 달라지도록 했다. 계수뿐만 아니라 해당 구간의 prediction error power도 합성에 반영한 것이다.

청크별로 계산한 $f_{10}(n)$ 의 RMS는 0.01622로, 전체 모델의 0.03198보다 작았다. 두 합성 결과는 각각

`syn_f10_chunk.wav`, `syn_noise_chunk.wav` 로 저장했다.

5. 청크별 결과 분석

시간-주파수 도메인 신호 분석

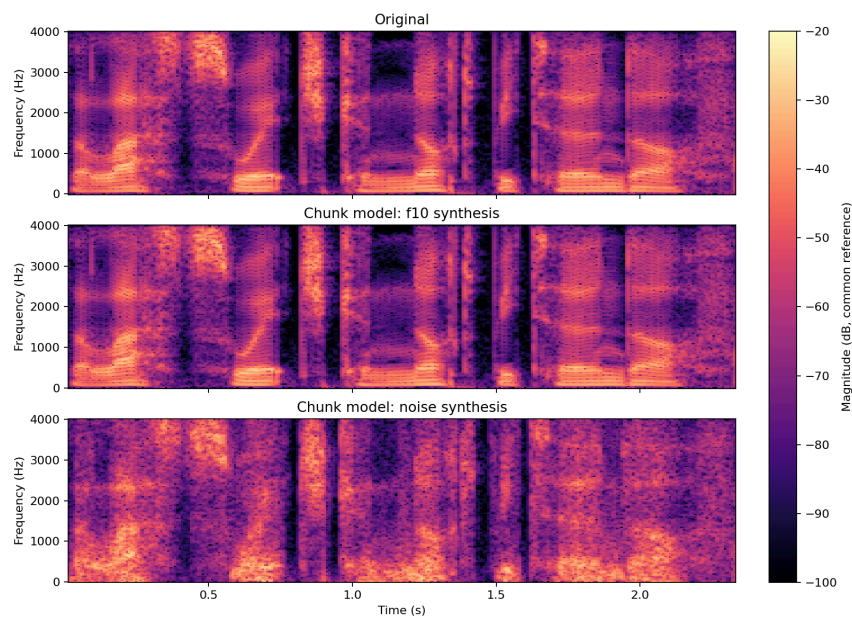


그림 2. 청크별 모델의 원본 및 두 개의 합성 결과

그림 2에서도 prediction error를 입력한 출력은 원본과 동일하게 나타났다. 최대 절댓값 오차는 약 2.22×10^{-16} 이었으며, WAV로 저장한 뒤에는 원본과 모든 샘플이 일치했다. Prediction error를 이용한 복원은 전체 모델과 청크별 모델에서 모두 가능했다.

White noise를 입력한 결과에는 차이가 있었다. 그림 1에서는 비슷한 주파수 분포가 계속 이어졌지만, 그림 2에서는 발화 구간에 따라 주파수별 에너지가 달라졌다. 특히 약 0.7~0.9초와 1.6~2.1초 부근의 저주파 에너지 분포가 원본의 변화와 비슷하게 나타났다. 발화 사이에 에너지가 작아지는 구간도 구분할 수 있었다. 두 그림은 같은 dB 범위로 표시하여 비교했다.

다만 원본 유성음에서 보이는 촘촘한 주기적 구조까지 복원되지는 않았다. 청크별로 필터를 바꾸어도 excitation은 여전히 white noise이므로, 원본의 피치 주기성까지 그대로 나타나지는 않는 것으로 생각할 수 있다. `syn_noise_chunk.wav` 을 직접 들어 본 결과, 원본과 같은 문장을 속삭이며 말하는 듯이 들렸다. 전체 모델의 출력에서는 잡음만 들렸지만, 청크별 출력에서는 단어를 식별할 수 있었다. 원본 화자의 목소리가 그대로 복원된 것은 아니지만, 발화 내용을 알아들을 수 있을 정도로 차이가 나타났다.

시간에 따른 에너지 변화

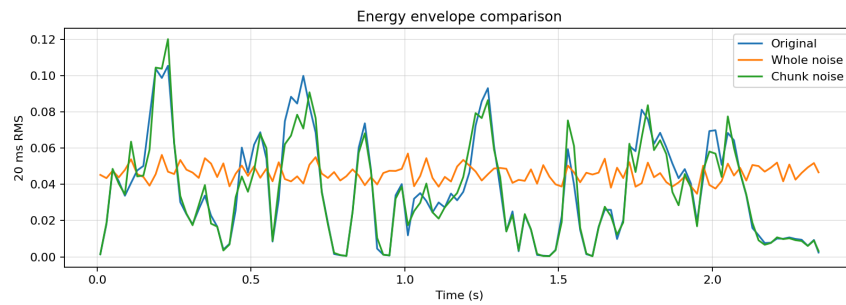


그림 3. 원본과 두 개의 잡음 합성 결과의 20ms RMS

에너지 변화를 확인하기 위해 20 ms마다 RMS를 계산했다. 마지막에 남은 10 ms 구간도 포함했다. 그림 3에서 전체 모델의 잡음 출력은 원본의 발화 구간과 관계없이 RMS가 비교적 일정하게 유지되었다. 반면 청크별 출력은 원본에서 에너지가 커지는 구간과 작아지는 구간을 비슷하게 따라갔다.

신호	전체 RMS	Peak amplitude	구간별 RMS 변동계수	원본 RMS 변화와의 상관계수
원본	0.04618	0.59949	0.748	1.000
전체 모델 잡음 출력	0.04609	0.17542	0.101	-0.024
청크별 모델 잡음 출력	0.04499	0.32040	0.744	0.963

위 표의 변동계수는 구간별 RMS의 표준편차를 평균으로 나눈 값이다. 전체 모델에서는 0.101로 작았지만, 청크별 모델에서는 0.744로 원본의 0.748과 비슷했다. 원본 RMS 변화와의 상관계수 역시 전체 모델에서는 -0.024, 청크별 모델에서는 0.963이었다. 그림 3에서 보였던 차이를 수치로도 확인할 수 있었다.

청크별 모델에서는 계수와 잡음의 세기를 함께 바꾸었으므로, 이 차이가 모두 계수 때문인지는 구분할 필요가 있었다. 이를 확인하기 위해 잡음 입력은 전체 모델과 동일하게 유지하고, 계수만 청크별로 바꾸어 보았다. 이때 RMS 변동계수는 0.663이었지만, 원본 RMS 변화와의 상관계수는 0.214였다. 계수를 바꾸는 것만으로도 출력에 변화가 생기지만, 원본의 에너지 변화를 따라가는 데에는 청크별 P_{10} 을 적용한 영향도 컸다.

즉, 청크별 모델에서는 구간마다 달라지는 주파수 특성과 excitation의 세기를 함께 반영하면서 원본의 시간적인 변화에 가까워졌다.

6. 결론

Prediction error를 입력하면 전체 모델과 청크별 모델 모두 원본을 복원할 수 있었다. 반면 white noise를 입력한 경우에는 청크별로 모델링했을 때 원본의 시간별 주파수 분포와 에너지 변화가 더 잘 나타났다. 전체 음성의 평균적인 특성을 구하는 것만으로는 발화에 따른 변화를 표현하기 어렵고, 이를 반영하려면 짧은 구간마다 모델을 구할 필요가 있음을 확인할 수 있었다. 다만 white noise를 excitation으로 사용할 경우 원본의 피치 주기성까지는 복원되지 않았다.

Repository

26-Fall-Adap/HW03 at main · Yejin-02/26-Fall-Adap

Contribute to Yejin-02/26-Fall-Adap development by creating an account on GitHub.

Yejin-02/26-Fall-Adap



<https://github.com/Yejin-02/26-Fall-Adap/tree/main/HW03>

1 Contributor 0 Issues 0 Stars 0 Forks

